

Remote Worker Setup & Management Guide

Remote Worker Setup & Management Guide

A comprehensive blueprint for secure networking, remote access, and persistent workflows.

Table of Contents

1. [Tailscale: Secure Mesh Networking](#)
2. [SSH: Secure Shell & Passwordless Entry](#)
3. [TMUX: Persistent “Never-Die” Sessions](#)
4. [The Recovery Workflow](#)
5. [GUI File Access \(SFTP & Samba\)](#)
6. [SSHFS: SSH Filesystem Mounting](#)

1. Tailscale: Secure Mesh Networking

Tailscale provides a secure overlay network, allowing you to access your worker from anywhere without port forwarding.

1.1 Installation

Run this on all machines (Master and Worker):

```
curl -fsSL https://tailscale.com | sh
sudo tailscale up
```

1.2 Advanced Routing (Subnet & Exit Nodes)

To use the worker as a bridge to a local network or as a VPN exit node:

Prerequisite: Enable IP Forwarding

```
echo 'net.ipv4.ip_forward = 1' | sudo tee -a /etc/sysctl.d/99-tailscale.conf
echo 'net.ipv6.conf.all.forwarding = 1' | sudo tee -a /etc/sysctl.d/99-tailscale.conf
sudo sysctl -p /etc/sysctl.d/99-tailscale.conf
```

Start with Specific Flags Note: Tailscale flags are “sticky” (saved between sessions). Use `--reset` to clear old configurations.

- **Subnet Router:** `bash sudo tailscale up --reset --advertise-routes=<YOUR_SUBNET_RANGE> # Example: 192.168.1.0/24`
- **Exit Node:** `bash sudo tailscale up --reset --advertise-exit-node`
- **Tailscale SSH:** (Keyless auth via Tailscale) `bash sudo tailscale up --ssh`

Critical Step: You must “Approve” these routes in the Tailscale Admin Console (<https://login.tailscale.com/admin/>) under **Machine Settings > Edit Route Settings**.

2. SSH: Secure Shell & Passwordless Entry

The standard for remote management. Configure this for secure, automated access.

2.1 Installation & Firewall (On Worker)

```
sudo apt update && sudo apt install openssh-server -y
sudo ufw allow ssh
sudo systemctl enable --now ssh
```

2.2 Standard Connection Syntax

To connect to a remote machine manually using its IP address or hostname:

```
ssh username@ip_address
# OR
ssh username@hostname
```

2.3 Passwordless Authentication (From Master)

Generate and copy ed25519 keys from your local machine:

```
ssh-keygen -t ed25519 -C "worker_key" # Press Enter for defaults
ssh-copy-id username@100.x.y.z # Use the Worker's Tailscale IP
```

2.4 SSH Config Aliases (~/.ssh/config)

Simplify your connection by creating an alias on your Master machine:

```
Host lab
  HostName 100.x.y.z
  User username
  IdentityFile ~/.ssh/id_ed25519
  ServerAliveInterval 60
```

Usage: You can now simply type `ssh lab` to connect.

3. TMUX: Persistent “Never-Die” Sessions

TMUX ensures that your remote processes continue running even if your connection drops.

3.1 Installation (On Worker Only)

```
sudo apt update && sudo apt install tmux -y
```

3.2 The “Immortal” Workflow

Always connect to your worker using this command to rejoin your existing session:

```
tmux attach || tmux
```

3.3 Essential Key Commands

All commands start with the prefix `Ctrl + b: - d` — **DETACH:** Leaves everything running in the

background. - % — Split screen **vertically**. - " — Split screen **horizontally**. - o — Cycle focus between panes.

4. The Recovery Workflow

If your connection blips or you move between networks: 1. **Re-connect via SSH:** `ssh lab` 2. **Re-attach to TMUX:** `tmux attach` 3. Your terminal environment is exactly as you left it.

5. GUI File Access (SFTP & Samba)

Efficiently manage remote files using graphical file managers (like Nautilus, Dolphin, or Finder).

5.1 SFTP (Secure File Transfer Protocol)

SFTP runs over SSH and is the most secure way to browse your worker's files in a GUI.

- **Protocol:** `sftp://`
- **Address Example:** `sftp://username@100.x.y.z` (Use the Tailscale IP from Section 2.3)
- **Automation:** If you configured SSH Config Aliases (Section 2.4), many Linux file managers can connect using the alias directly: `sftp://lab`.

5.2 Samba (SMB) Access via Subnet Routing

If your worker is on a local network with SMB shares, use Tailscale's Subnet Routing to access them securely from anywhere.

- **Prerequisite:** Ensure Section 1.2 is configured and routes are **Approved** in the Tailscale Admin Console.
 - **Protocol:** `smb://`
 - **Address Example:** `smb://192.168.12.1/` (Use the local LAN IP of the target machine).
-

6. SSHFS: SSH Filesystem Mounting

SSHFS Supplement

SSHFS (Secure Shell FileSystem) is a FUSE-based client that allows you to securely mount a remote directory onto your local machine via an SSH connection. This enables you to interact with remote files as if they were stored locally, fixing permission issues and allowing you to use remote filesystems or mountpoints seamlessly and natively on your local machine.

6.1 Installation (On Master Machine)

```
sudo apt update && sudo apt install sshfs -y
```

6.2 Basic Usage & Examples

Create a local mount point and link the remote directory. You can use an SSH alias, a direct IP address, or the standard `username@host` syntax.

Example 1: Using an SSH Config Alias (Section 2.4)

```
mkdir -p ~/mnt/worker
sshfs lab:/home/username ~/mnt/worker -o reconnect,idmap=user,allow_other
```

Example 2: Using a direct IP address (Local or Tailscale)

```
sshfs 192.168.13.1:/home/username ~/mnt/worker -o reconnect,idmap=user,allow_other
```

Example 3: Using the full Username & IP Address syntax

```
sshfs target-username@target-ipaddr:/home/username ~/mnt/worker -o
reconnect,idmap=user,allow_other
```

6.3 Key Flags Breakdown

- `-o reconnect`: Automatically attempts to reconnect if the network connection drops.
- `-o idmap=user`: Maps remote UID/GID to local user (fixes "Permission Denied" errors).
- `-o allow_other`: Allows other users/processes to access the filesystem mount.

6.4 Unmounting (Standard & Lazy)

When finished, safely disconnect the filesystem. If the mount is "busy" (files open or terminal inside the directory), use the Lazy Unmount flags to detach it immediately and clean up as soon as it's free.

Standard Unmount:

```
fusermount -u ~/mnt/worker
# OR
sudo umount ~/mnt/worker
```

Lazy Unmount (Use if "Device is busy"):

- **FUSE Method:** `bash fusermount -u -z ~/mnt/worker`
- **Standard Method:** `bash sudo umount -l ~/mnt/worker`